

A Proposal to add stack trace library

Significant changes to [P0881R0](#) are marked with blue.

☐ Show deleted lines from P0881R0.

I. Motivation

In the current working draft [[N4741](#)] there is no way to get, store and decode the current call sequence. Such call sequences are useful for debugging and post mortem debugging. They are popular in other programming languages (like Java, C#, Python).

Pretty often assertions can't describe the whole picture of a bug and do not provide enough information to locate the problem. For example, you can see the following message on out-of-range access:

```
boost/array.hpp:123: T& boost::array<T, N>::operator[](boost::array<T, N>::size_type): Assertion '(i < N)&&("out of range")' failed.  
Aborted (core dumped)
```

That's not enough information in the assert message to locate the problem without debugger.

This paper proposes classes that could simplify debugging and may change the assertion message into the following:

```
Expression 'i < N' is false in function 'T& boost::array<T, N>::operator[](boost::array<T, N>::size_type) [with T = int; long unsigned i  
Backtrace:  
0# boost::assertion_failed_msg(char const*, char const*, char const*, char const*, long) at ../example/assert_handler.cpp:39  
1# boost::array<int, 5ul>::operator[](unsigned long) at ../../boost/array.hpp:124  
2# bar(int) at ../example/assert_handler.cpp:17  
3# foo(int) at ../example/assert_handler.cpp:25  
4# bar(int) at ../example/assert_handler.cpp:17  
5# foo(int) at ../example/assert_handler.cpp:25  
6# main at ../example/assert_handler.cpp:54  
7# 0x00007F991FD69F45 in /lib/x86_64-linux-gnu/libc.so.6  
8# 0x000000000401139
```

II. Impact on the Standard

This proposal is a pure library extension and it does not break the existing code and does not degrade performance. It does not require any changes in the core language and could be implemented in the standard C++.

III. Design Decisions

The design is based on the Boost.Stacktrace library, a popular library that does not depend on any non-standard library components.

Note about signal safety: this proposal does not attempt to provide a signal-safe solution for capturing and decoding stacktraces. Such functionality currently is not implementable on some of the popular platforms. However, the paper attempts to provide extensible solution, that may be made signal safe some day by providing a signal safe allocator and changing the stacktrace implementation details.

Note on performance: during Boost.Stacktrace development phase many users requested a fast way to store stack trace, without decoding the function names. This functionality is preserved in the paper. All the `stack_frame` functions and constructors are lazy and won't decode the pointer information if there was no explicit request from class user.

Note on allocations: initial implementations of Boost.Stacktrace were not using allocator and all the frames were placed inside a fixed size internal storage. That was a mistake! Sometimes the most important information is located at the bottom of the stack. For example if you run Boost.Test, then the test name will be located low on the stack. With a fixed size storage the bottom of the stack could be lost along with the information.

Current design assumes that by default users wish to see the whole stack and OK with dynamic allocations, because do not construct stacktrace in performance critical places. For those users, who wish to use stacktrace on a hot path or in embedded environments `basic_stacktrace` allows to provide a custom allocator that allocates on the stack or in some other place, where users thinks it is appropriate.

Note on returning `std::string` and not having `noexcept` on `stack_frame::source_line()`: Unfortunately this is a necessity on some platforms, where getting source line requires allocating or where source file name returned into a storage provided by user.

Note on expected implementation: We assume that Standard Library implementations would allow to disable/enable gathering stack traces by a compiler switch that does not require recompiling the whole project. In other words, we expect to see a compiler option like `-fno-stacktrace` or `libstacktrace/lib_stacktrace_noop` libraries with the same ABI that would force the constructor of the `basic_stacktrace` to do nothing. This feature is implemented in Boost.Stacktrace and is highly requested in big projects.

IV. Proposed Interface

Header `<stacktrace>` synopsis

```

namespace std {

    class stack_frame;

    template<typename Allocator>
    class basic_stacktrace;

    // free functions

    // This is the alias to use unless you'd like to provide a specific allocator to basic_stacktrace.
    using stacktrace = basic_stacktrace<allocator<stack_frame>>;

    // Outputs stacktrace in a human readable format to output stream.
    template<typename CharT, typename TraitsT, typename Allocator>
    basic_ostream< CharT, TraitsT > & operator<<(basic_ostream<CharT, TraitsT>& os, const basic_stacktrace<Allocator>& bt);

    // Outputs stacktrace in a human readable format to string.
    template<typename Allocator>
    string to_string(const basic_stacktrace<Allocator>& f);

    // Outputs frame in a human readable format to string.
    string to_string(const stack_frame& f);

    // Outputs frame in a human readable format to output stream.
    template<typename CharT, typename TraitsT>
    basic_ostream< CharT, TraitsT > & operator<<(basic_ostream<CharT, TraitsT>& os, const stack_frame& f);
}

```

Class stack_frame

The stack_frame class stores a pointer to function and allows querying information about that function.

```

namespace std {
    class stack_frame {
    public:
        using native_frame_ptr_t = unspecified;

        // construct/copy/destruct
        constexpr stack_frame() noexcept;
        constexpr stack_frame(const stack_frame&) noexcept = default;
        constexpr stack_frame& operator=(const stack_frame&) noexcept = default;

        constexpr explicit stack_frame(native_frame_ptr_t f) noexcept;
        template<typename T> explicit stack_frame(T* address) noexcept;

        constexpr native_frame_ptr_t address() const noexcept;
        constexpr explicit operator bool() const noexcept;

        constexpr strong_ordering operator <=>(const stack_frame& rhs) = default;

        // functions that query information about stored address
        string name() const;
        string source_file() const;
        size_t source_line() const;

    private:
        native_frame_ptr_t data; // exposition only
    };
}

```

stack_frame constructors

`stack_frame() noexcept;`

Constructs stack_frame that references nullptr address. Calls to source_file() and source_line() will return empty string. Calls to source_line() will return 0.

`explicit stack_frame(native_frame_ptr_t addr) noexcept;`
`template<typename T> explicit stack_frame(T * addr) noexcept;`

Constructs stack_frame that references addr and could later generate information about that address using platform specific features.

stack_frame member functions

`std::string name() const;`

Returns empty string or platform specific name of the function that the stored address is pointing to. Throws std::bad_alloc if not enough memory to construct resulting string.

`constexpr native_frame_ptr_t address() const noexcept;`

Returns address stored by this.

`std::string source_file() const;`

Returns empty string or path to the source file, where the function of the frame is defined. Returns empty string if this->source_line() == 0. Throws std::bad_alloc if not enough memory to construct resulting string.

```
std::string source_line() const;
```

Returns 0 or code line in the source file, where the function of the stored address is defined. Throws std::bad_alloc if not enough memory to construct resulting string.

```
explicit operator bool() const noexcept;
```

Returns true if this stores and address other than NULL.

Class template <basic_stacktrace>

The basic_stacktrace template class stores current call sequence on construction and provides functions for querying information about the call sequence.

```
namespace std {
    template<typename Allocator>
    class basic_stacktrace {
    public:
        using value_type = stack_frame;
        using const_reference = const value_type &;
        using size_type = implementation-defined;
        using const_iterator = implementation-defined;
        using allocator_type = Allocator;

        // functions that capture current call sequence
        basic_stacktrace() noexcept;
        explicit basic_stacktrace(const allocator_type& a) noexcept;
        basic_stacktrace(size_type skip, size_type max_depth, const allocator_type& a = allocator_type()) noexcept;

        // construct/copy/destruct
        basic_stacktrace(const basic_stacktrace &);
        basic_stacktrace(basic_stacktrace &&) noexcept;
        basic_stacktrace & operator=(const basic_stacktrace &);
        basic_stacktrace & operator=(basic_stacktrace &&) noexcept;
        ~basic_stacktrace();

        // public member functions
        size_type size() const noexcept;
        const_reference operator[](size_type ) const noexcept;
        const_iterator begin() const noexcept;
        const_iterator end() const noexcept;

        explicit operator bool() const noexcept;

        template <typename Allocator2>
        strong_ordering operator <=>(const basic_stacktrace< Allocator2 >& rhs) noexcept = default;

    private:
        vector<value_type>    stack_frames; // exposition only
    };
}
```

basic_stacktrace constructors

```
basic_stacktrace() noexcept;
explicit basic_stacktrace(const allocator_type & a) noexcept;
```

Stores the current call sequence inside *this without querying information about each call.

Any exception raised during this operation is silently ignored. In case of exception (bool)*this is false

```
basic_stacktrace(size_type skip, size_type max_depth, const allocator_type& a = allocator_type()) noexcept;
```

Stores [skip; skip + max_depth) of the current function call sequence inside *this without querying information about each call.

Any exception raised during this operation is silently ignored. In case of exception (bool)*this is false

basic_stacktrace member functions

```
const_reference operator[](size_type frame_no) const noexcept;
```

Returns frame that references the actual frame info, stored inside *this.

Parameters: frame_no - zero-based index of frame to return. 0 is the function index where stacktrace was constructed and index close to this->size() contains function main().

```
explicit operator bool() const noexcept;
```

Returns true if call sequence was successfully stored.

V. Feature-testing macro

For the purposes of SG10 we recommend the feature-testing macro name `__cpp_lib_stacktrace`.